Reason. Plan. Act. Physical AI Workshop

Lecture 3a: Integrated Task and Motion Planning

Sarah Keren

The Taub Faculty of Computer Science
Technion - Israel Institute of Technology



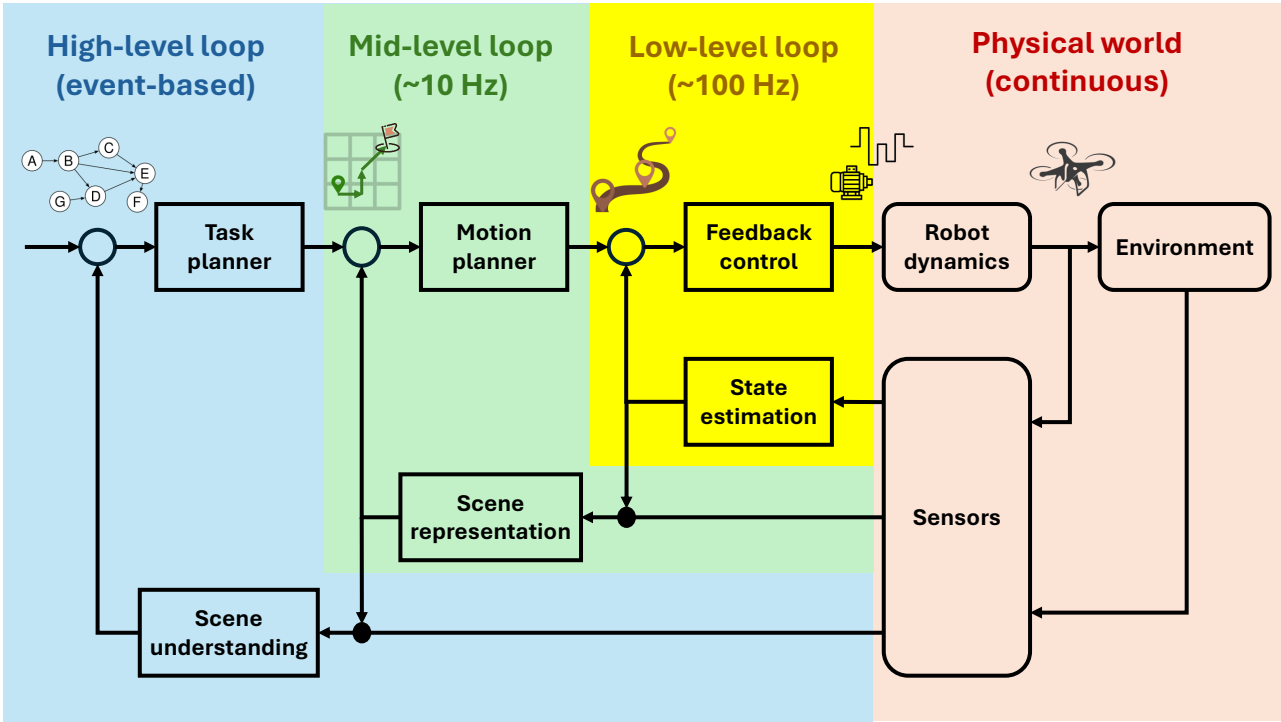


’Pick up the tennis ball’

’Pick up an apple’

’I am hungry’

Layered Decision-Making



Why Task Planning Alone Is Not Enough

Task planning assumes:

- Actions are discrete and deterministic
- Preconditions and effects fully describe feasibility

Why Task Planning Alone Is Not Enough

Task planning assumes:

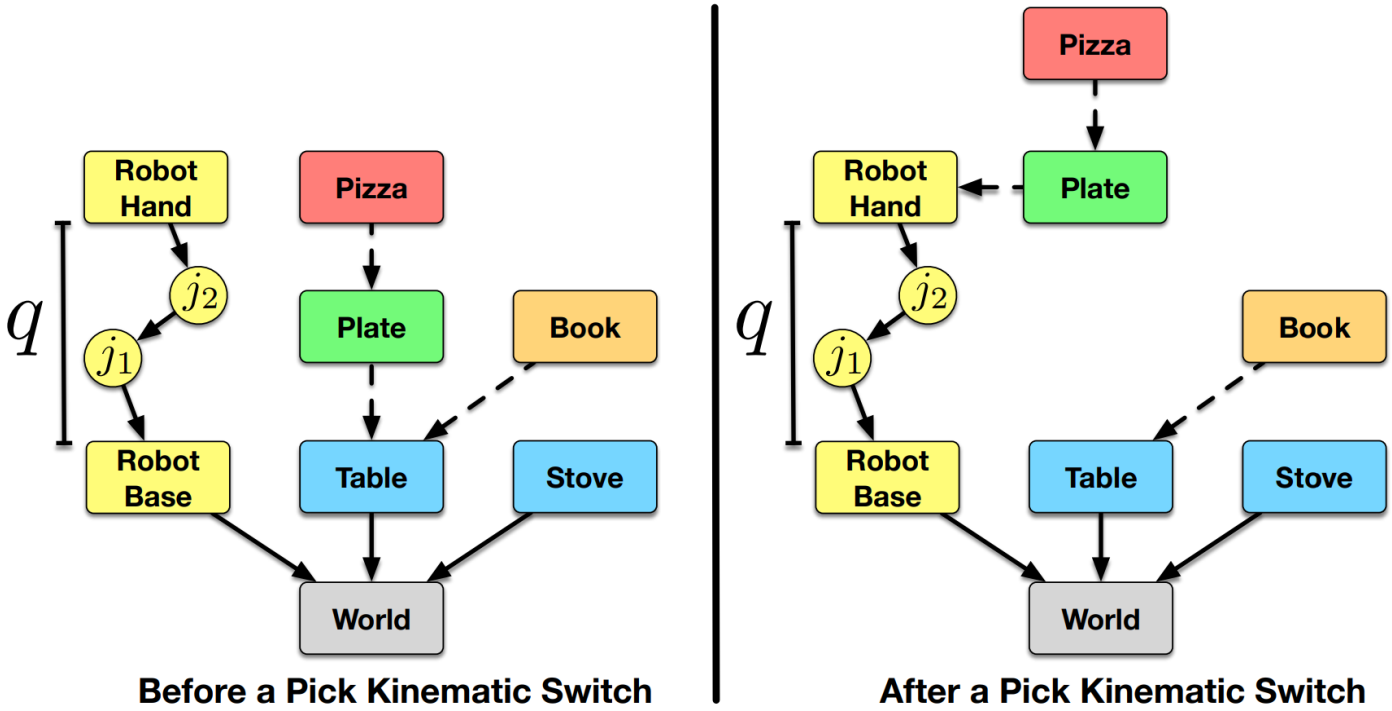
- Actions are discrete and deterministic
- Preconditions and effects fully describe feasibility

In robotics:

- Collision-free motions may not exist
- Grasps and placements are geometric
- Feasibility depends on continuous variables

A valid symbolic plan may be impossible to execute

Multi-Modal Planning



A kinematic tree for a kitchen environment with a robot manipulator with two joints (j1, j2), a fixed Table and Stove, and movable objects Plate, Pizza, and Book.

Why is multi-modal planning not enough?

Algorithmic Paradigms for TMP

Three main paradigms:

1. Sequence then Satisfy
2. Satisfy then Sequence
3. Interleaved Task and Motion Planning

Different points in the design space trade off *feasibility guidance* vs. *search flexibility*.

Sequence Then Satisfy

Approach:

- First generate a task plan (discrete)
- Then attempt to instantiate motions (continuous)

Sequence Then Satisfy

Approach:

- First generate a task plan (discrete)
- Then attempt to instantiate motions (continuous)

Pros:

- Leverages classical planning directly
- Clear separation of concerns

Sequence Then Satisfy

Approach:

- First generate a task plan (discrete)
- Then attempt to instantiate motions (continuous)

Pros:

- Leverages classical planning directly
- Clear separation of concerns

Cons:

- Many symbolic plans are geometrically infeasible
- Motion planning failures occur late (wasted search)

Satisfy Then Sequence

Approach:

- Sample continuous parameters first (grasps, poses, IK)
- Build a discrete plan around feasible samples

Satisfy Then Sequence

Approach:

- Sample continuous parameters first (grasps, poses, IK)
- Build a discrete plan around feasible samples

Pros:

- Early geometric feasibility filtering

Satisfy Then Sequence

Approach:

- Sample continuous parameters first (grasps, poses, IK)
- Build a discrete plan around feasible samples

Pros:

- Early geometric feasibility filtering

Cons:

- Blind sampling can be inefficient
- Hard to scale without strong priors / heuristics

Interleaved Task and Motion Planning

Approach:

- Alternate between symbolic reasoning and geometric checks
- Use constraints to guide and prune task search *early*

Interleaved Task and Motion Planning

Approach:

- Alternate between symbolic reasoning and geometric checks
- Use constraints to guide and prune task search *early*

Key principles:

- **Fail fast**: discover infeasibility early
- **Lazy evaluation**: defer expensive checks until needed
- **Reuse**: cache IK / collision / motion results

Interleaved Task and Motion Planning

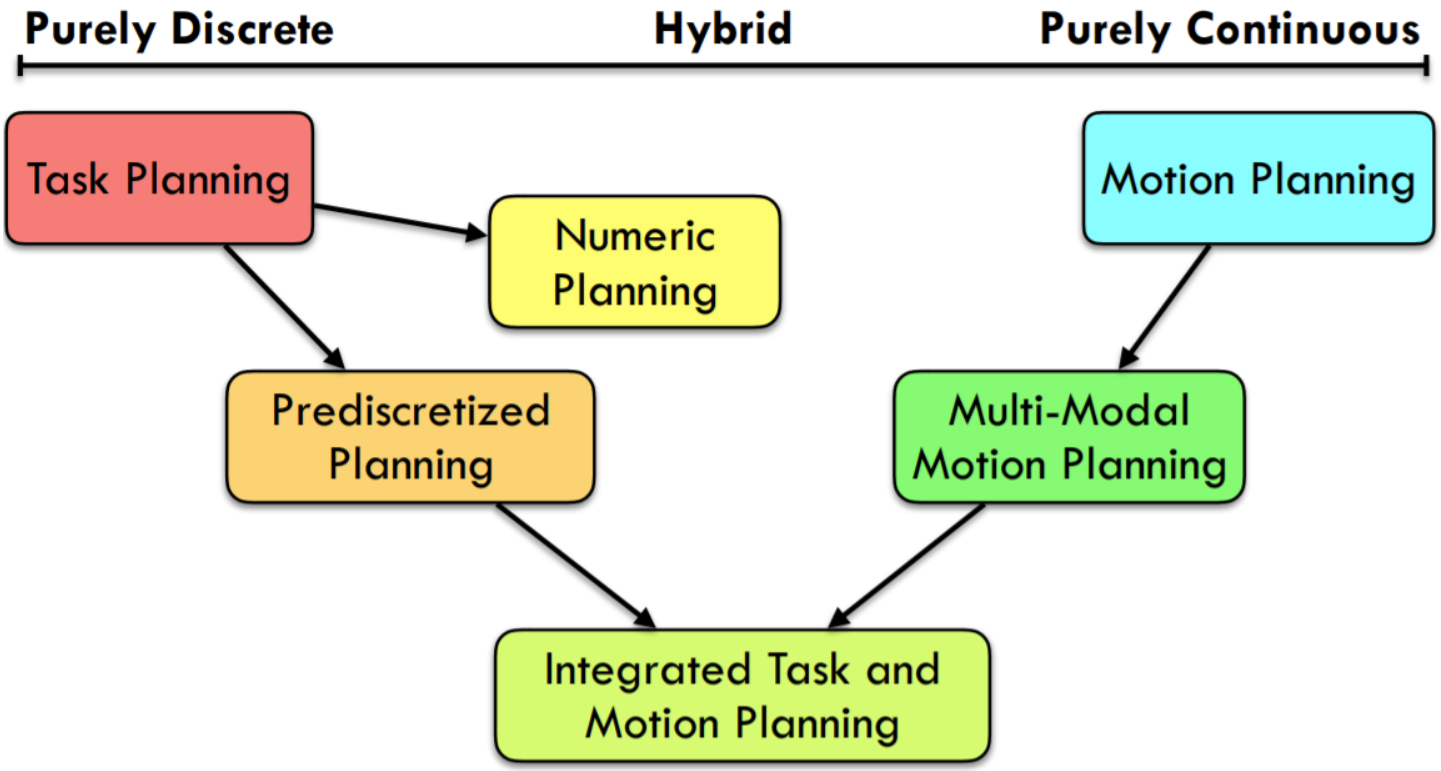
Approach:

- Alternate between symbolic reasoning and geometric checks
- Use constraints to guide and prune task search *early*

Key principles:

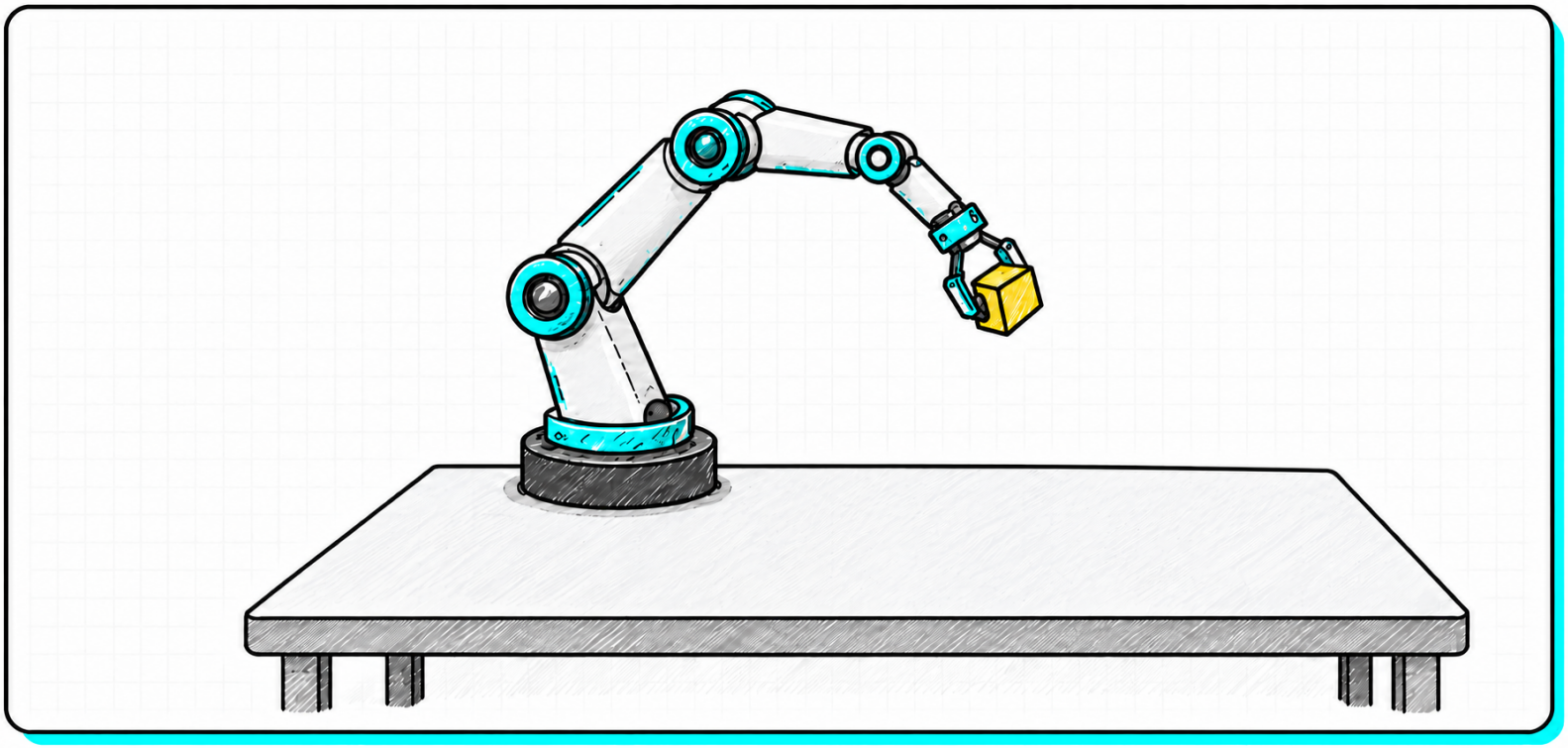
- **Fail fast**: discover infeasibility early
- **Lazy evaluation**: defer expensive checks until needed
- **Reuse**: cache IK / collision / motion results

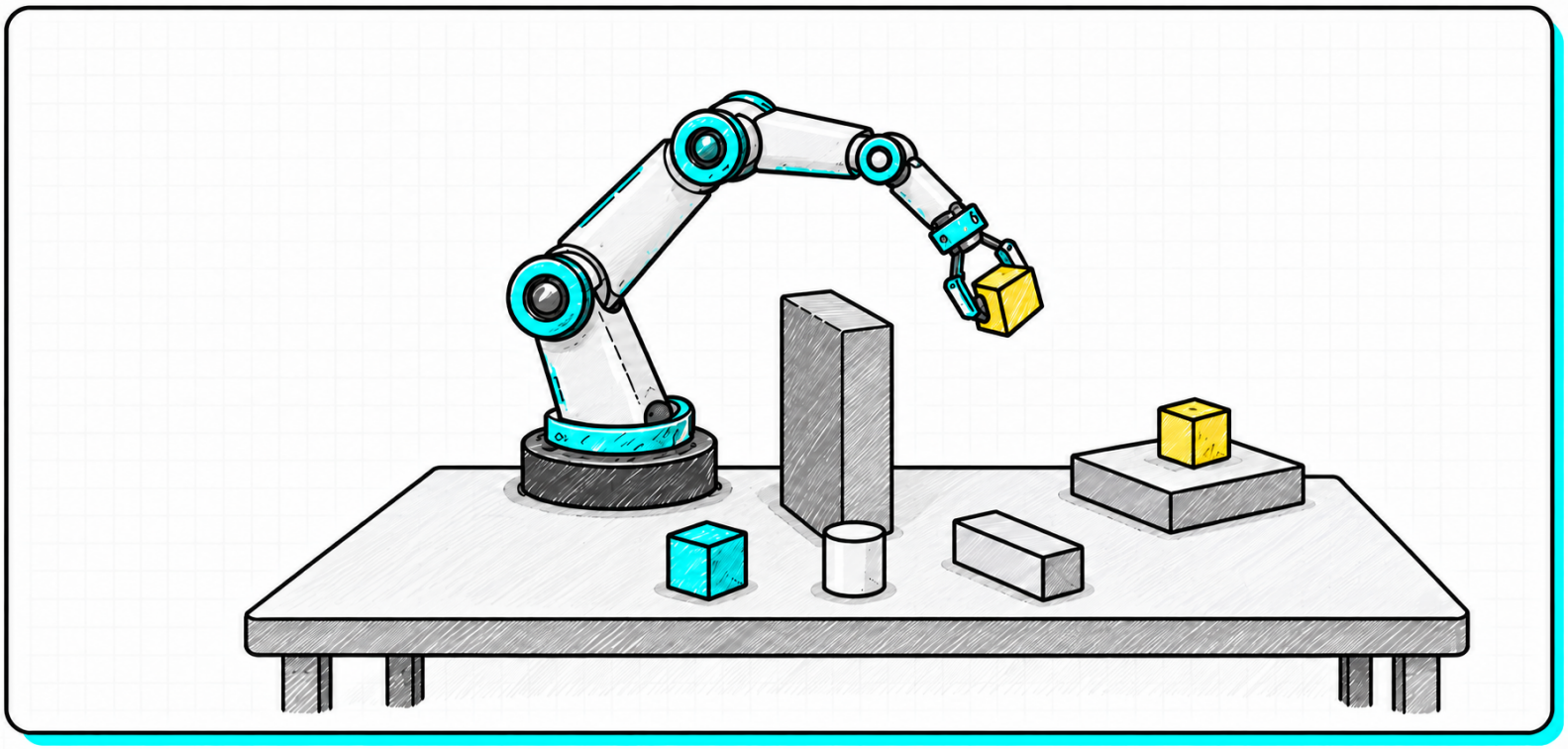
This is the dominant modern approach in practical systems

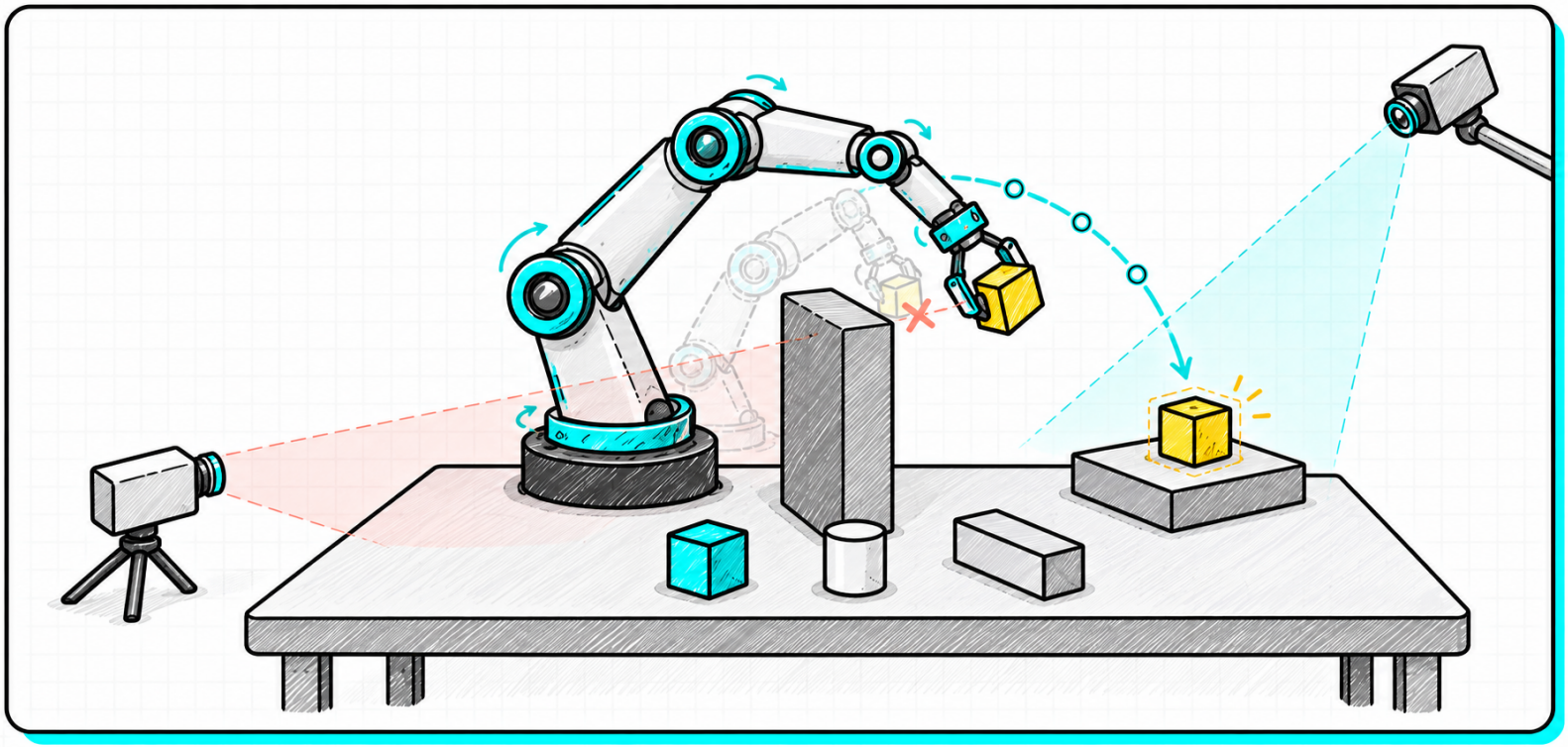

Purely Discrete
Hybrid
Purely Continuous
Task Planning
Numeric Planning
Motion Planning
Prediscretized Planning
Multi-Modal Motion Planning
Integrated Task and Motion Planning

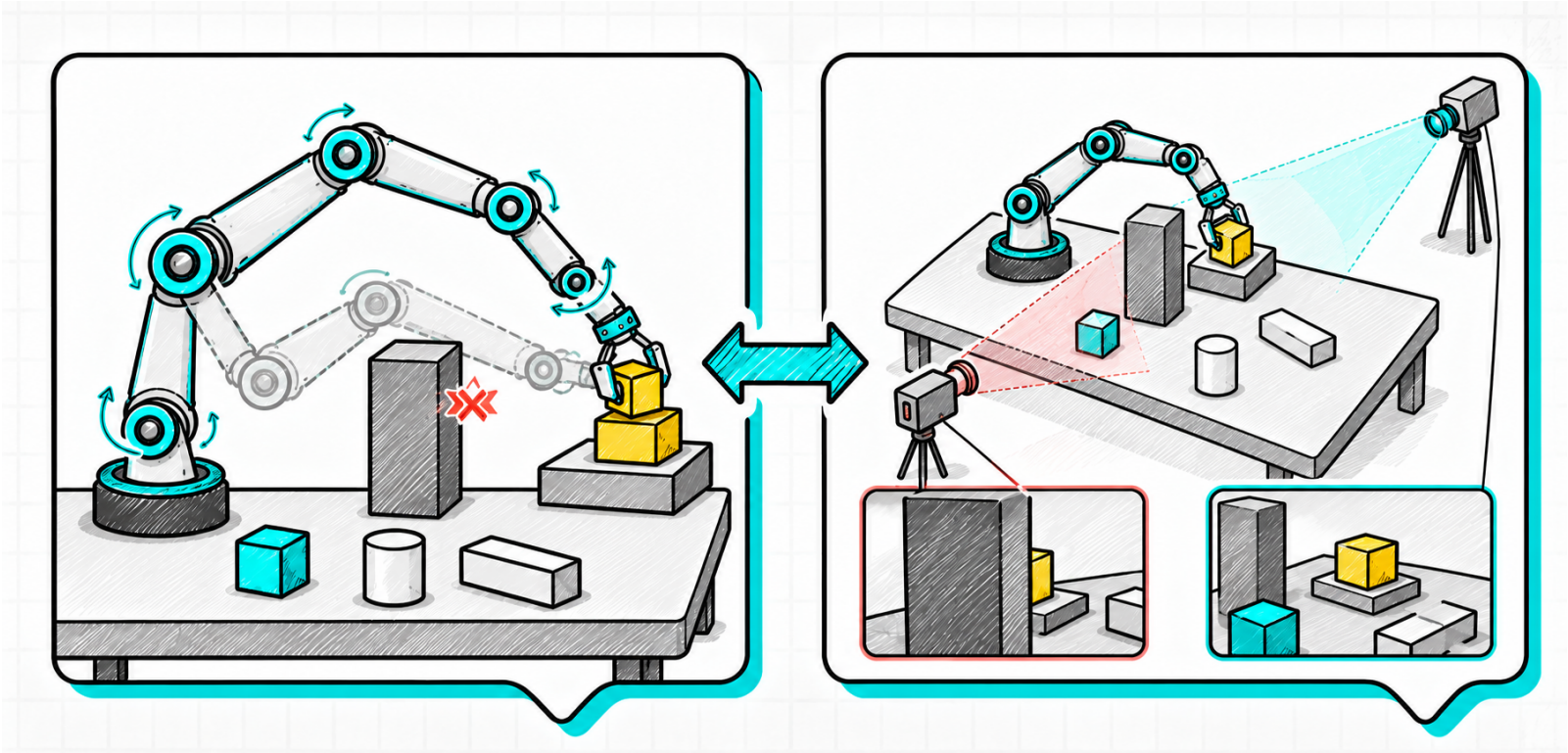
Ideas ?

Modeling Uncertainty









Uncertainty in Robotics

Uncertainty in Robotics

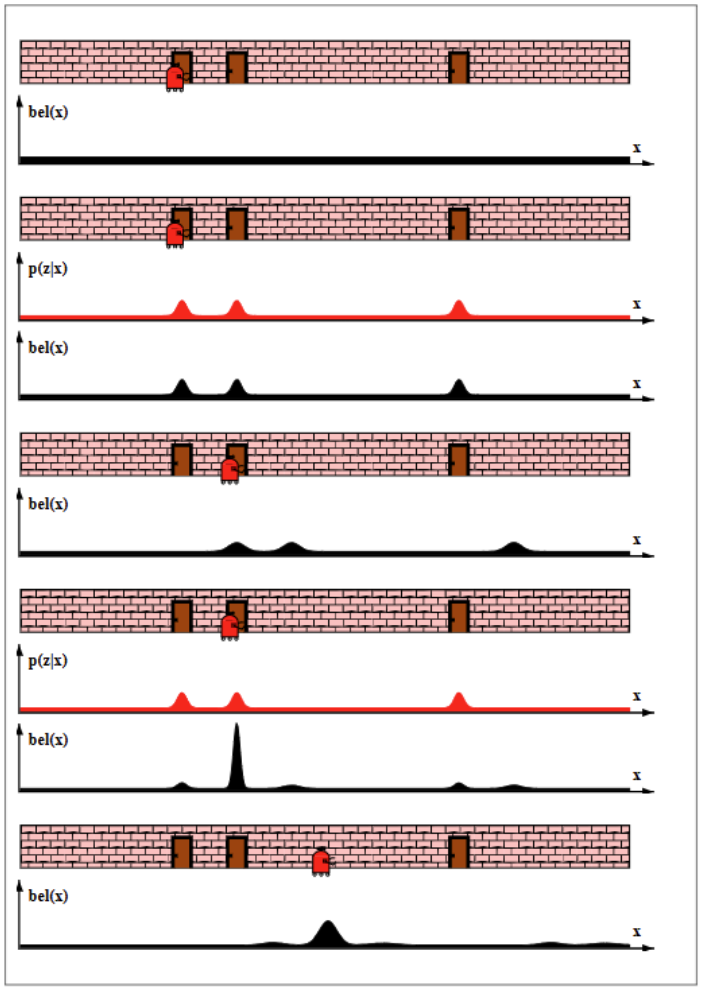
- Robot environments are inherently unpredictable (especially if people are around)
- Sensors are limited in what they can perceive
- Actuators [somewhat] unpredictable (especially for low-cost and mobile robots)
- Modeling uncertainty - all models are abstractions

Uncertainty in Robotics

- We are looking for ways to explicitly account for uncertainty, instead of relying on a single ”best guess”.
- We want decisions to be robust
- We want robots to actively choose to reduce uncertainty when this is the superior choice.

Example - Markov Localization

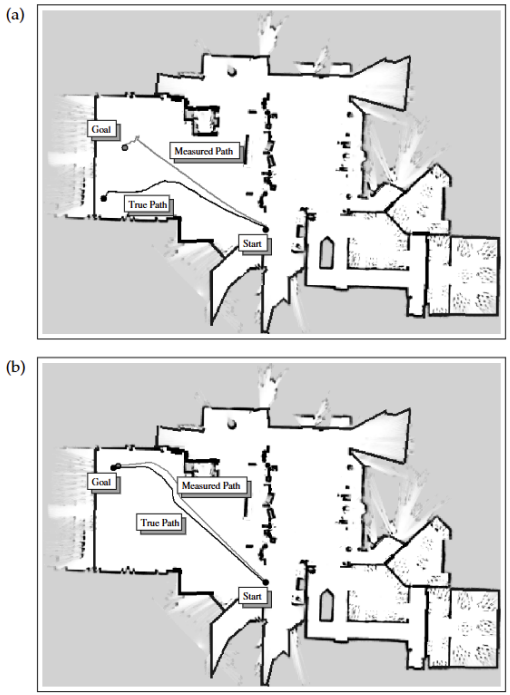
The agent is given a map of an environment with 3 indistinguishable doors.
Initially, it does not know where it is and needs to localize itself on the map using its sensor data.



From Probabilistic Robotics S. Thrun, W. Burgard, and D. Fox, Probabilistic Robotics. Cambridge, MA: MIT Press, 2005.

Example - Coastal Navigation

In addition to accounting for monetary uncertainty, an agent can anticipate future uncertainty and plan accordingly.
Estimated and actual paths differ when the agent uses features (or landmarks) from the environment.



By Nicholas Roy MIT

What are the models we know for modeling uncertainty?

What are the models we know for modeling uncertainty?

Various models for various asepcts of uncertainty.

Markov Decision Process (MDP)

A **Markov Decision Process** (MDP) is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma \rangle$ where

- $\mathcal{S}$ is a finite set of states (states are described via a set of random variables $\mathcal{X} = X_1, \ldots, X_n$).
- $\mathcal{A}$ is a finite set of actions
- $\mathcal{P} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is the **transition probability function**, where $\mathcal{P}(s' \mid s, a) = p(S_{t+1} = s' \mid S_t = s, A_t = a)$ denotes the probability of transitioning to state s' after taking action a in state s.
- $\mathcal{R} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the **reward function**, where $\mathcal{R}(s, a)$ (or its expectation) gives the immediate reward obtained after taking action a in state s, and
- **optional:** γ is a discount factor $\gamma \in [0, 1]$ that is used to favor immediate rewards over future rewards.

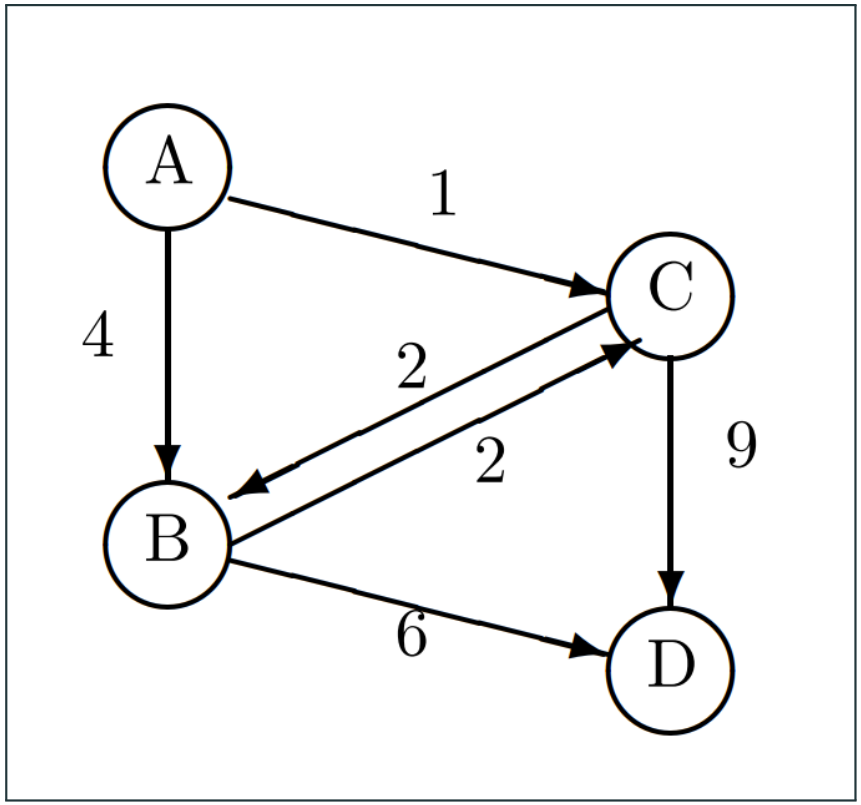
Partially Observable Markov Decision Process (POMDP)

A **Partially Observable Markov Decision Process**(POMDP) is a tuple $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma, \Omega, \mathcal{O}, \beta_0 \rangle$ where

- $\mathcal{S}$, $\mathcal{A}$, $\mathcal{P}$, $\mathcal{R}$ and γ are as for an MDP.
- Ω is a set of observations (observation tokens),
- $\mathcal{O} : \Omega \times \mathcal{S} \times \mathcal{A} \to [0, 1]$ is a sensor function specifying the conditional probability $\mathcal{O}(o|s, a) = p(O_{t+1} = o|S_t = s, A_t = a)$ of receiving observation $o \in \Omega$ in state s after applying a [1].
- β_0 the initial belief: a probability distribution over the states.

[1] alternatively: $\mathcal{O}(o|s) = p(O_t = o|S_t = s)$

We know that the model for A* and best-first-search is a graph.



What is the underlying model for an MDP and POMDP?

Key Concepts

- State x_t, control/action u_t, sensor measurement z_t.
- Belief: $\beta(x_t) = p(x_t \mid z_{1:t}, u_{1:t})$.
- **Markov assumption:** given x_t, past and future are conditionally independent.
- Two-step update: prediction (motion) + correction (observation).

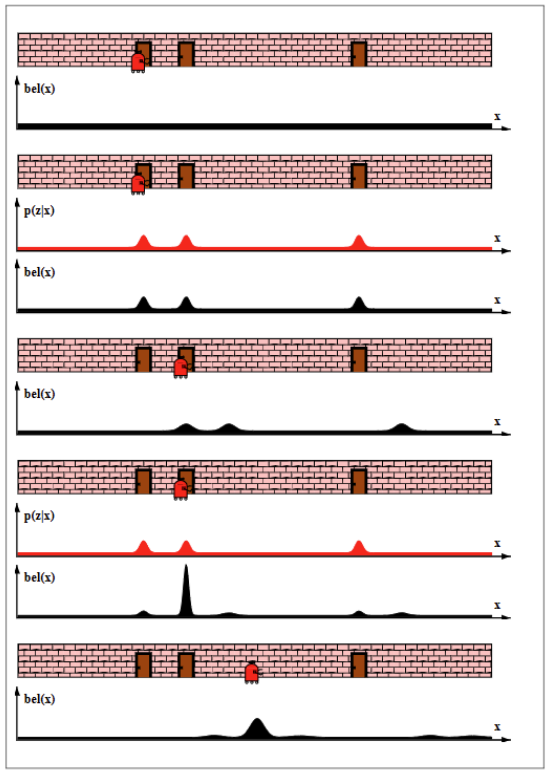
Why recursive state estimation? Why maintain a belief?

Why recursive state estimation? Why maintain a belief?

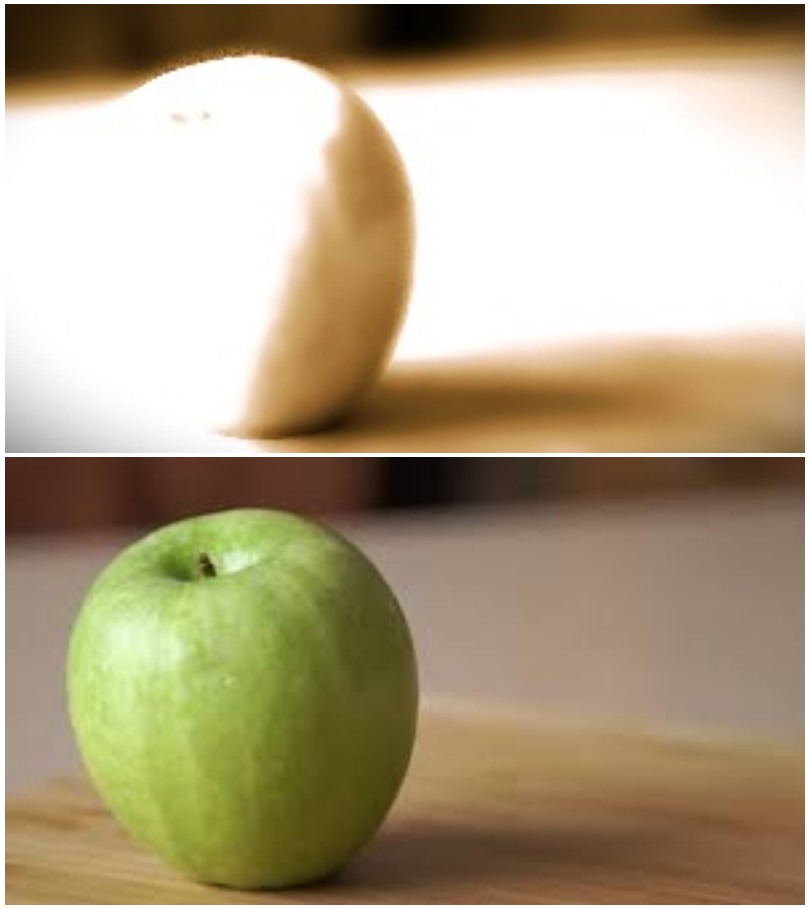
- Robots operate in uncertain, dynamic environments.
- Instead of relying on a “single best guess” state, we maintain distributions and make robust decisions.

Intuition: Prediction vs. Correction

- **Prediction:** motion mode typically increases uncertainty.
- **Correction:** sensor model typically reduces uncertainty; shifts belief towards states consistent with measurement.
- **Underlying idea:** alternate these steps to yield an updated belief.



Partial Observability



From Full to Partial Observability

In classical and stochastic planning, we assumed the agent has access to the **true state**.

More precisely, **the agent observes all relevant features needed for decision-making**.

This assumption is called **full observability**.

In reality,

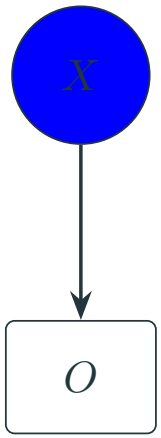
- the true state is often **hidden**
- the agent receives **partial and noisy observations**

Reminder: Observation / Sensor Model

Instead of observing the current state directly, the agent receives $Oervation_t$ generated from the hidden state via the sensor model

$$\Pr(Oervation_t \mid x_t)$$

The sensor model captures sensor noise, limited sensing range or resolution, ambiguity (multiple states producing similar observations), etc.



Partial Observability Over Time

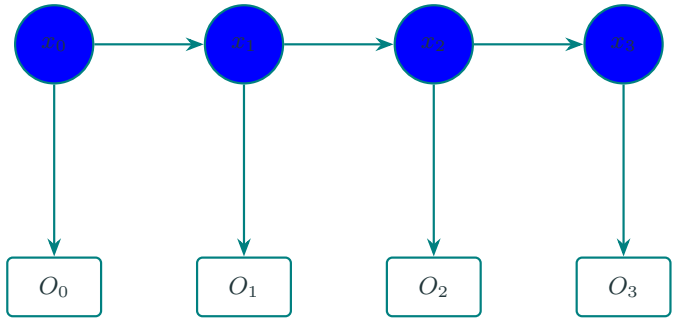
$t = 3$



$t = 15$



Partial Observability Over Time



- When the environment is dynamic, over time, the agent observes only the **observation sequence** $O_{0:T} = (O_0, O_1, \ldots, O_T)$, while the **state sequence** $x_{0:T} = (x_0, x_1, \ldots, x_T)$ remains unobserved.
- At each time step, we now have **two** coupled processes:
  - **State dynamics:** $x_t \sim \Pr(\cdot \mid x_{t-1})$
  - **Observation generation:** $O_t \sim \mathcal{O}(\cdot \mid x_t)$

Partial Observability Over Time

- Modeling assumptions:
  - Markov dynamics:

$$\Pr(x_{t+1} \mid x_{0:t}) = \Pr(x_{t+1} \mid x_t)$$

  - Memoryless observations:

$$\Pr(O_t \mid x_{0:t}, O_{0:t-1}) = \Pr(O_t \mid x_t)$$

- Under these assumptions, the following conditional independencies hold:

$$x_{t+1} \perp\!\!\!\perp x_{0:t-1} \mid x_t, \qquad O_t \perp\!\!\!\perp (O_{0:t-1}, x_{0:t-1}) \mid x_t$$

- Therefore, the joint distribution factors as:

$$\Pr(x_{0:T}, O_{0:T}) = \Pr(x_0) \prod_{t=0}^{T-1} \Pr(x_{t+1} \mid x_t) \prod_{t=0}^{T} \Pr(O_t \mid x_t)$$

This structure defines a Hidden Markov Model (HMM).

Agency under partial observability

Recall that the environment-agent interactions give rise to a sequence or trajectory: $s_0, a_0, r_1, s_1, a_1, r_2, s_2, a_2, r_3, ...$

From the point of view of the partially informed agent, we have a **history**: $O_0, a_0, r_1, O_1, a_1, r_2, O_2, a_2, r_3, ...$

The agent needs to maintain its belief based on the current (possibly null) observation.

Recursive Bayesian Estimation

Starting point:

$$\beta(x_{t-1}) = p(x_{t-1} \mid z_{1:t-1}, u_{1:t-1})$$

Recursive Bayesian Estimation

Starting point:

$$\beta(x_{t-1}) = p(x_{t-1} \mid z_{1:t-1},\, u_{1:t-1})$$

Prediction:

$$\underbrace{p(x_t \mid z_{1:t-1},\, u_{1:t})}_{\text{prior}} = \int p(x_t \mid u_t, x_{t-1})\ \underbrace{p(x_{t-1} \mid z_{1:t-1},\, u_{1:t-1})}_{\beta(x_{t-1})}\ dx_{t-1}$$

Recursive Bayesian Estimation

Starting point:

$$\beta(x_{t-1}) = p(x_{t-1} \mid z_{1:t-1},\, u_{1:t-1})$$

Prediction:

$$\underbrace{p(x_t \mid z_{1:t-1},\, u_{1:t})}_{\text{prior}} = \int p(x_t \mid u_t, x_{t-1})\ \underbrace{p(x_{t-1} \mid z_{1:t-1},\, u_{1:t-1})}_{\beta(x_{t-1})}\ dx_{t-1}$$

Correction using observation:

$$\underbrace{\beta(x_t)}_{\text{posterior}} = \eta\, p(z_t \mid x_t)\ \underbrace{p(x_t \mid z_{1:t-1},\, u_{1:t})}_{\text{prior}}$$

Bayes Filter

Algorithm `Bayes_filter`$(\beta(x_{t-1}), u_t, z_t)$

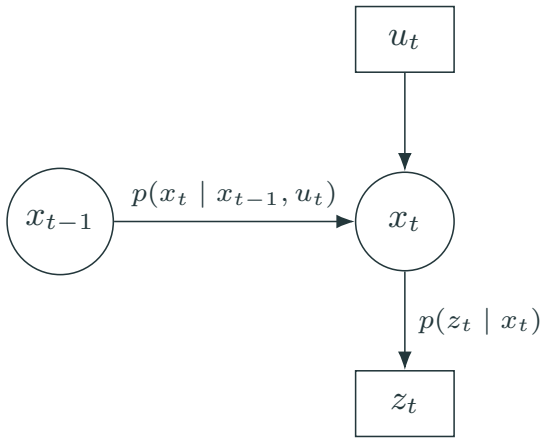
forall x_t do

$$\overline{\beta}(x_t) = \int p(x_t \mid u_t, x_{t-1})\, \beta(x_{t-1})\, dx$$

$$\beta(x_t) = \eta\, p(z_t \mid x_t)\, \overline{\beta}(x_t)$$

end

return $\beta(x_t)$



Defining TMP

What Is a TMP Problem?

A TMP problem involves:

- Discrete actions (pick, place, open, move)
- Continuous parameters (poses, grasps, trajectories)
- Geometric and kinematic constraints

What Is a TMP Problem?

A TMP problem involves:

- Discrete actions (pick, place, open, move)
- Continuous parameters (poses, grasps, trajectories)
- Geometric and kinematic constraints

Goal:

- Find a sequence of actions *and* continuous parameters such that
- all constraints are satisfied

What Is a TMP Problem?

A TMP problem involves:

- Discrete actions (pick, place, open, move)
- Continuous parameters (poses, grasps, trajectories)
- Geometric and kinematic constraints

Goal:

- Find a sequence of actions *and* continuous parameters such that
- all constraints are satisfied

Search occurs in a mixed discrete–continuous space

Ways to Define a Task and Motion Planning (TMP) Problem

(High-Level View) Task and Motion Planning (TMP) concerns finding a sequence of high-level actions together with continuous motions such that the task is achieved and all physical constraints are satisfied.

This general view translates into many definitions and approaches.

What Is a TMP Problem?

1. TMP as a Two-Level Planning Problem
   - Discrete task planning over symbolic actions
   - Continuous motion planning for each action
   - A task plan is valid only if all motions are feasible

What Is a TMP Problem?

2. TMP as Planning in a Hybrid State Space

- States include both discrete and continuous components

$$s = (s^{\text{disc}}, s^{\text{cont}})$$

- Actions induce symbolic effects and geometric constraints

What Is a TMP Problem?

3. TMP as Constrained Search

- Search over symbolic plans
- Motion planning enforces feasibility constraints

$$\text{Find } \pi \quad \text{s.t. motions}(\pi) \text{ are feasible}$$

What Is a TMP Problem?

4. TMP with Implicit Feasibility (Streams)

- Continuous parameters generated on demand
- Feasibility checked lazily via procedural generators
- Avoids full discretization of continuous geometry

5. TMP as a Decision-Theoretic Problem

- Uncertainty in state, perception, or action outcomes
- Actions may fail due to geometric or sensing uncertainty
- Objective defined over expected cumulative reward

$$\max_{\pi} \mathbb{E}\left[\sum_{t} \gamma^t r(s_t, a_t)\right]$$

What Is a TMP Problem?

TMP is not a single formalism, but a family of problem definitions that trade off expressiveness, scalability, and realism.

Formal Definition of TMP #1

A **Task and Motion Planning (TAMP)** problem is a tuple:

$$\mathcal{P}_{\text{TAMP}} = \langle \mathcal{S}, \mathcal{A}, \mathcal{X}, \mathcal{T}, \mathcal{C}, s_0, G \rangle$$

- $\mathcal{S}$ — discrete (symbolic) state space
- $\mathcal{A}$ — discrete actions (pick, place, move, open)
- $\mathcal{X}$ — continuous parameters (poses, grasps, trajectories)
- $\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{X} \to \mathcal{S}$ — transition model
- $\mathcal{C}(s, a, x)$ — geometric and kinematic constraints
- s_0 — initial symbolic state
- $G \subseteq \mathcal{S}$ — goal states

Formal Definition of TMP #1

A **Task and Motion Planning (TAMP)** problem is a tuple:

$$\mathcal{P}_{\text{TAMP}} = \langle \mathcal{S}, \mathcal{A}, \mathcal{X}, \mathcal{T}, \mathcal{C}, s_0, G \rangle$$

- $\mathcal{S}$ — discrete (symbolic) state space
- $\mathcal{A}$ — discrete actions (pick, place, move, open)
- $\mathcal{X}$ — continuous parameters (poses, grasps, trajectories)
- $\mathcal{T} : \mathcal{S} \times \mathcal{A} \times \mathcal{X} \to \mathcal{S}$ — transition model
- $\mathcal{C}(s, a, x)$ — geometric and kinematic constraints
- s_0 — initial symbolic state
- $G \subseteq \mathcal{S}$ — goal states

Objective: find $(a_1, x_1), \ldots, (a_T, x_T)$ such that

$$s_T \in G \quad \text{and} \quad \mathcal{C}(s_t, a_t, x_t) \text{ holds for all } t.$$

Formal Definition of TMP #2

A **Belief-Space Task and Motion Planning** problem is defined as:

$$\mathcal{P} = \langle \mathcal{S}, \mathcal{A}, \mathcal{X}, \mathcal{B}, \mathcal{T}, \mathcal{O}, \mathcal{C}, b_0, G \rangle$$

- $\mathcal{S}$ — underlying discrete (symbolic) state space
- $\mathcal{A}$ — discrete high-level actions (pick, place, move, open)
- $\mathcal{X}$ — continuous parameters (poses, grasps, trajectories)
- $\mathcal{B}$ — belief space over symbolic and continuous variables

$$\mathcal{B} = \Delta(\mathcal{S} \times \mathcal{X})$$

- $\mathcal{T}(s' \mid s, a, x)$ — stochastic transition model
- $\mathcal{O}(o \mid s', a, x)$ — observation model
- $\mathcal{C}(s, a, x)$ — geometric, kinematic, and physical constraints
- $\beta_0 \in \mathcal{B}$ — initial belief over states and continuous parameters
- $G \subseteq \mathcal{S}$ — symbolic goal condition

Formal Definition of TMP #2

A **Belief-Space Task and Motion Planning** problem is defined as:

$$\mathcal{P} = \langle \mathcal{S}, \mathcal{A}, \mathcal{X}, \mathcal{B}, \mathcal{T}, \mathcal{O}, \mathcal{C}, b_0, G \rangle$$

- $\mathcal{S}$ — underlying discrete (symbolic) state space
- $\mathcal{A}$ — discrete high-level actions (pick, place, move, open)
- $\mathcal{X}$ — continuous parameters (poses, grasps, trajectories)
- $\mathcal{B}$ — belief space over symbolic and continuous variables

$$\mathcal{B} = \Delta(\mathcal{S} \times \mathcal{X})$$

- $\mathcal{T}(s' \mid s, a, x)$ — stochastic transition model
- $\mathcal{O}(o \mid s', a, x)$ — observation model
- $\mathcal{C}(s, a, x)$ — geometric, kinematic, and physical constraints
- $\beta_0 \in \mathcal{B}$ — initial belief over states and continuous parameters
- $G \subseteq \mathcal{S}$ — symbolic goal condition

Formal Definition of TMP #2

Find a policy

$$\pi : \mathcal{B} \rightarrow \mathcal{A} \times \mathcal{X}$$

that selects both:

- *what* action to take (task level)
- *how* to execute it (motion level)

Formal Definition of TMP #2

Find a policy

$$\pi : \mathcal{B} \rightarrow \mathcal{A} \times \mathcal{X}$$

that selects both:

- *what* action to take (task level)
- *how* to execute it (motion level)

- Reach goal states with high probability
- Satisfy constraints $\mathcal{C}(s, a, x)$
- Explicitly reason about uncertainty

Problems?

Constraint Semantics in Belief-Space TMP

In belief space, constraints cannot be Boolean — they must aggregate over uncertainty

Robust

$$\forall s \in \mathrm{supp}(\beta): \quad \mathcal{C}(s,a,x)$$

Safe for all possible worlds. Used for safety-critical execution.

Chance-Constrained

$$\mathbb{P}_{s\sim\beta}\big[\mathcal{C}(s,a,x)\big] \;\geq\; 1-\delta$$

Violation allowed with small probability. Used in Risk-aware planning

Soft / Expected Cost

$$\mathbb{E}_{s\sim\beta}\big[c_{\text{viol}}(s,a,x)\big] \;\leq\; \epsilon$$

Interpretation: Violations incur cost.Used for optimization with CMDPs.

Constraint Semantics in Belief-Space TMP

In belief space, constraints cannot be Boolean — they must aggregate over uncertainty

Robust

$$\forall s \in \mathrm{supp}(\beta): \quad \mathcal{C}(s,a,x)$$

Safe for all possible worlds. Used for safety-critical execution.

Chance-Constrained

$$\mathbb{P}_{s\sim\beta}\big[\mathcal{C}(s,a,x)\big] \;\geq\; 1-\delta$$

Violation allowed with small probability. Used in Risk-aware planning

Soft / Expected Cost

$$\mathbb{E}_{s\sim\beta}\big[c_{\text{viol}}(s,a,x)\big] \;\leq\; \epsilon$$

Interpretation: Violations incur cost.Used for optimization with CMDPs.

Formal Definition of TMP #3

TMP Definition by Kavraki et al.

A **task domain** is defined as a tuple $\Sigma = \langle S, s_o, S_G, A, \mathcal{P}, \mathcal{C} \rangle$ where

- S is a state space
- $s_o \in S$ is the initial state
- $S_G \subseteq S$ is a set of goal states
- $A(s) \subseteq A$ is the set of actions applicable at state s
- $\mathcal{P}$ is a transition function:
  - **Deterministic:** $\mathcal{P}(s, a) \in S$
  - **Stochastic:** $\mathcal{P}(s, a, s') \in [0, 1]$
- $\mathcal{C}(a, s)$ is a positive cost of performing action at state s (we can also have a reward function).

Formal Definition of TMP #3

TMP Definition by Kavraki et al.

A **task domain** is defined as a tuple $\Sigma = \langle S, s_o, S_G, A, \mathcal{P}, \mathcal{C} \rangle$ where

- S is a state space
- $s_o \in S$ is the initial state
- $S_G \subseteq S$ is a set of goal states
- $A(s) \subseteq A$ is the set of actions applicable at state s
- $\mathcal{P}$ is a transition function:
  - **Deterministic:** $\mathcal{P}(s, a) \in S$
  - **Stochastic:** $\mathcal{P}(s, a, s') \in [0, 1]$
- $\mathcal{C}(a, s)$ is a positive cost of performing action a at state s (we can also have a reward function).

Does it make sense to consider deterministic transition functions in robotics?

Formal Definition of TMP #3

A **motion domain** is defined by a configuration space $\mathcal{C}$.

Denote by $\mathcal{C}_{free} \subseteq \mathcal{C}$ the set of physically plausible configurations in which multiple bodies fo not occupy the same space.

A *motion plan* from initial configuration $q_0 \in \mathcal{C}$ to goal configuration $q_g \in \mathcal{C}$ is a sequence of configurations $(q_0, q_1, ..., q_m = q_g)$ where for all i,

$$\{\alpha q_{i-1} + (1-\alpha)q_i | \alpha \in [0,1]\} \subseteq \mathcal{C}_{free}$$

.

In other words, a motion plan is a sequence of configurations that can be traversed in straight lines within $\mathcal{C}_{free}$.

Formal Definition of TMP #3

A **task and motion planning domain** is defined as $\langle \Sigma, \mathcal{C}, \varphi, \xi, q_0 \rangle$ where

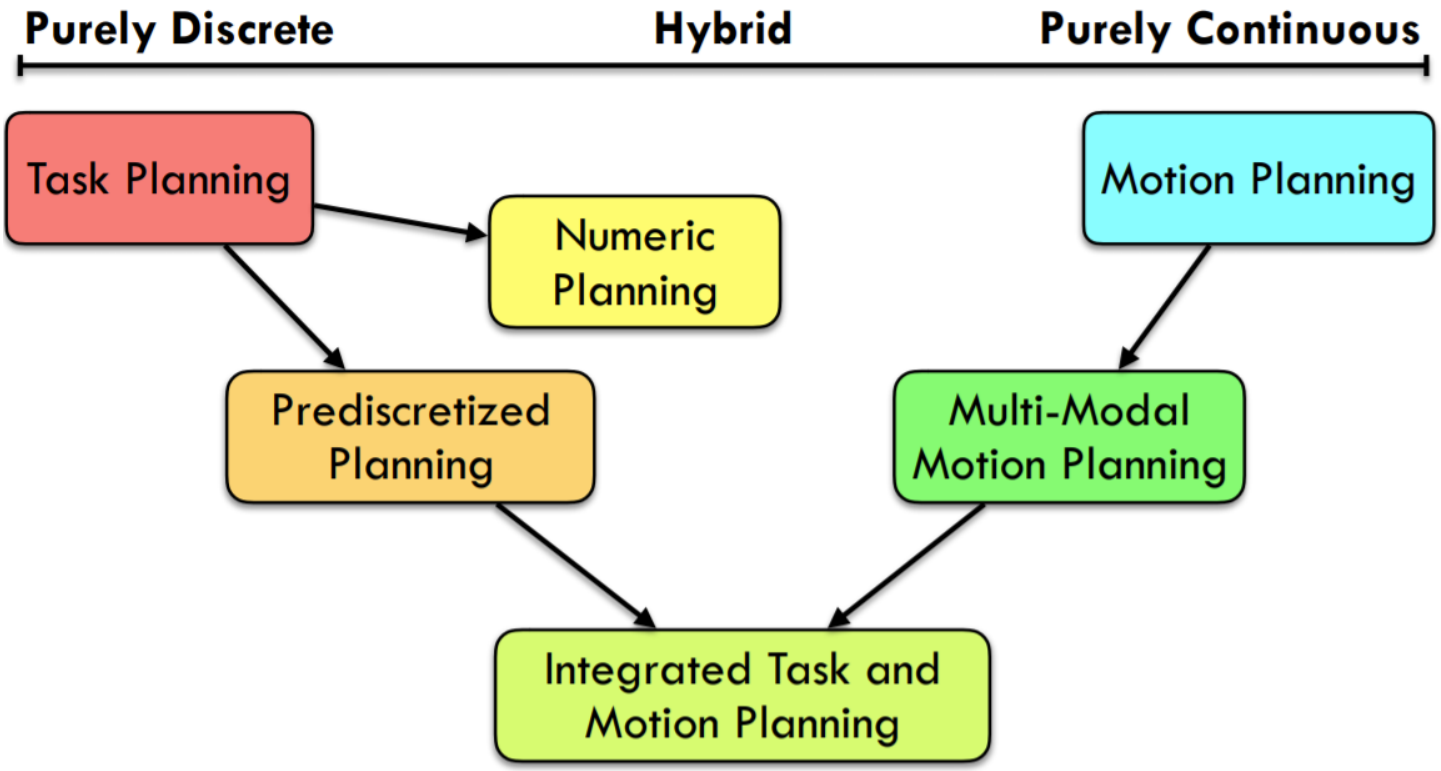
- $\Sigma = \langle S, s_o, S_G, A, \mathcal{P}, \mathcal{C} \rangle$ is a task domain,
- $\mathcal{C}$ is a world configuration space,
- $\varphi : \mathcal{C} \to S$ is a **state estimator function** that maps configurations to task domain states,
- $\xi : A \times \mathcal{C} \to \mathcal{C}^+$ is an **action mapper function** that maps actions into motion plans ($\mathcal{C}^+$ is the set of finite configuration sequences), and
- q_0 is an initial world configuration.

A TMP domain is a task domain and a motion domain with an initial configuration, linked via the state estimator (from motion to task) and action mapper (from task to motion).

Formal Definition of TMP #3

Denote by $\xi(a, q)_{-1}$ the final configuration in motion plan generated by action planner ξ given action aand state s.

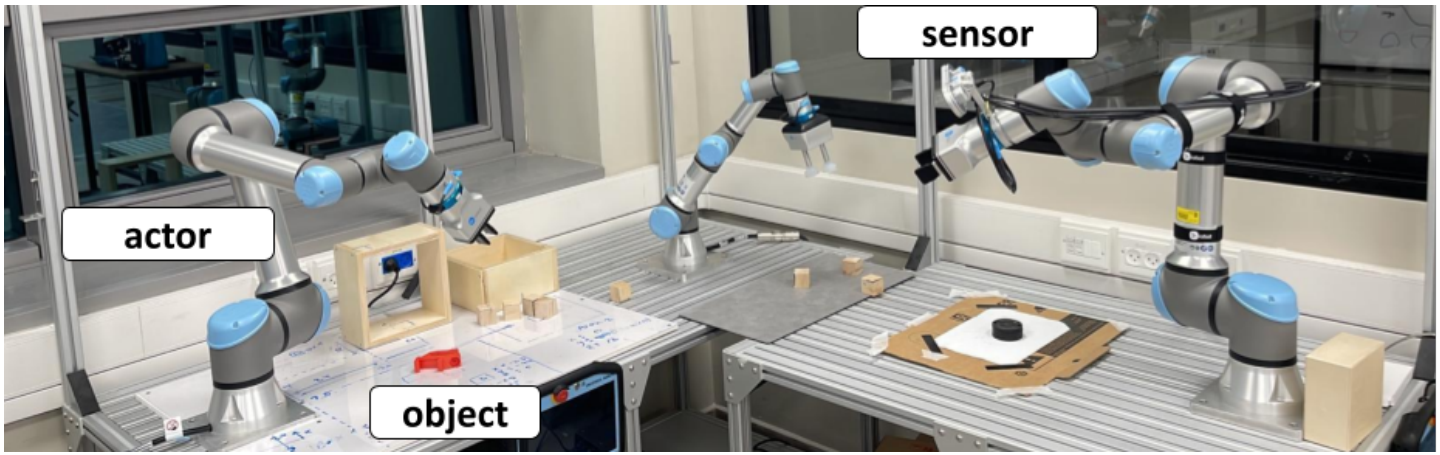
A **task and motion plan** is a task plan $(1, ..., n)$ such that a_1 is applicable from $\varphi(q_0)$, a_2 is applicable from $\varphi(\xi(1, q_0)_{-1})$, a_3 is applicable from $\varphi(\xi(a_1, \xi(a_1, q_0)_{-1}))$, etc., and the final configuration achieved in this manner q_g is mapped to a goal state, i.e., $\varphi(q_g) \in S_G$.


Purely Discrete
Hybrid
Purely Continuous
Task Planning
Numeric Planning
Motion Planning
Prediscretized Planning
Multi-Modal Motion Planning
Integrated Task and Motion Planning

Ideas ?

CLAIR lab Examples

From Beliefs to Decisions



Value of Assistance for Grasping

Mohammad Masarwy, Yuval Goshen, David Dovrat, Sarah Keren

https://arxiv.org/abs/2310.14402

Belief Update

When receiving an observation o, the agent updates its current belief β using its **belief update function** $\tau : \mathcal{B} \times \Omega \times \mathcal{S} \mapsto \mathcal{B}$ which maps belief $\beta \in \mathcal{B}$ to the new belief.

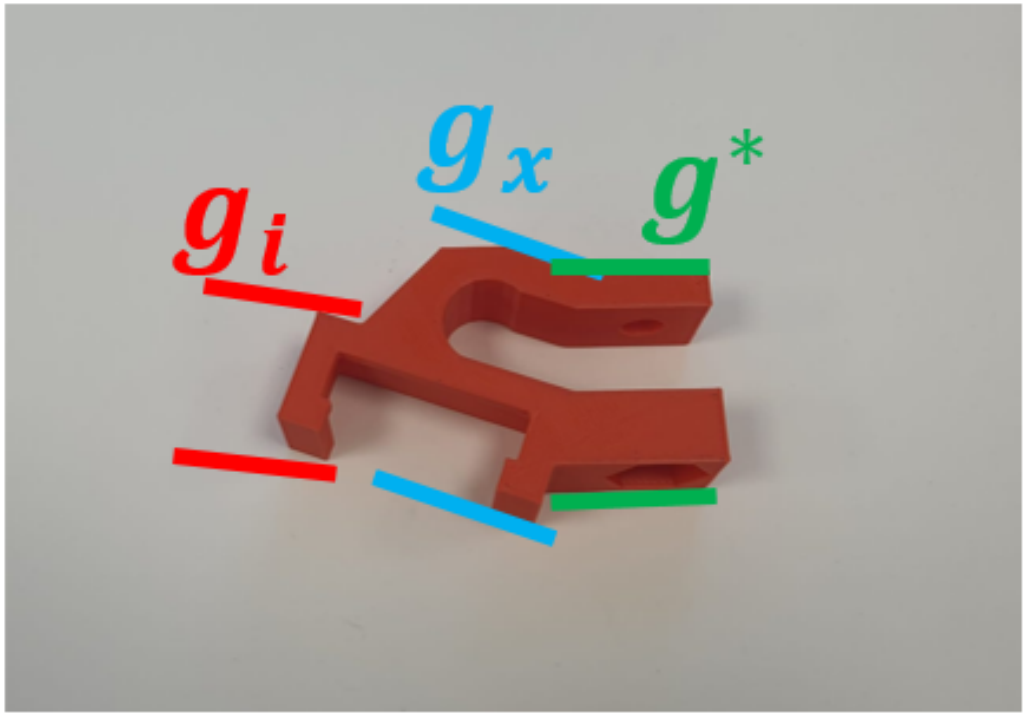
Commonly, a **Bayesian filter** is used:

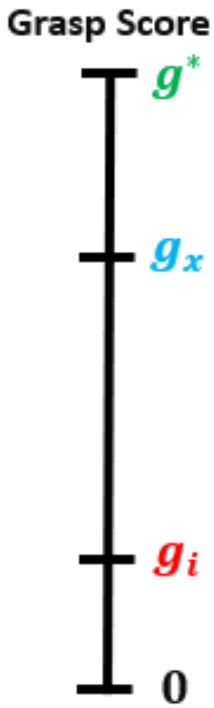
$$\beta^{o,a} = \frac{\mathcal{P}(o|s,a)\,\beta(s)}{\int_{s' \in \mathcal{S}} \mathcal{P}(o|s',a)\,\beta(s')\,ds'} \tag{1}$$

where $\mathcal{P}$ is the sensor function and $\beta(s)$ is the estimated probability that s is the actual world state
Discrete version:

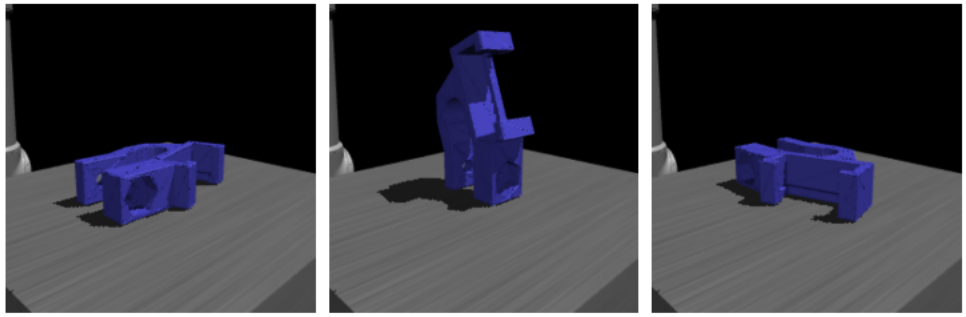
$$\beta^{o,a} = \frac{\mathcal{P}(o|s,a)\,\beta(s)}{\Sigma_{p' \in \mathcal{P}_o} \mathcal{P}(o|s',a)\,\beta(s')} \tag{2}$$

From Beliefs to Decisions



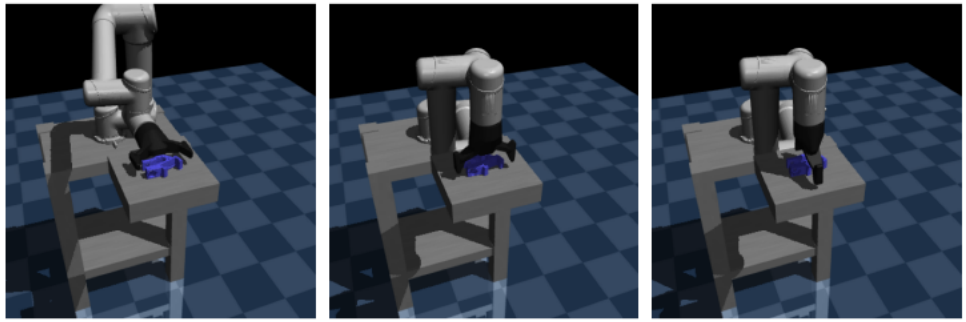


From Beliefs to Decisions



(a) (b) (c)

Fig. 2: Example stable poses.



(a) (b) (c)

Fig. 3: Example grasp configurations from which the actor can attempt to grasp the object - each configuration is associated with a score, i.e., probability of success.

From Beliefs to Decisions

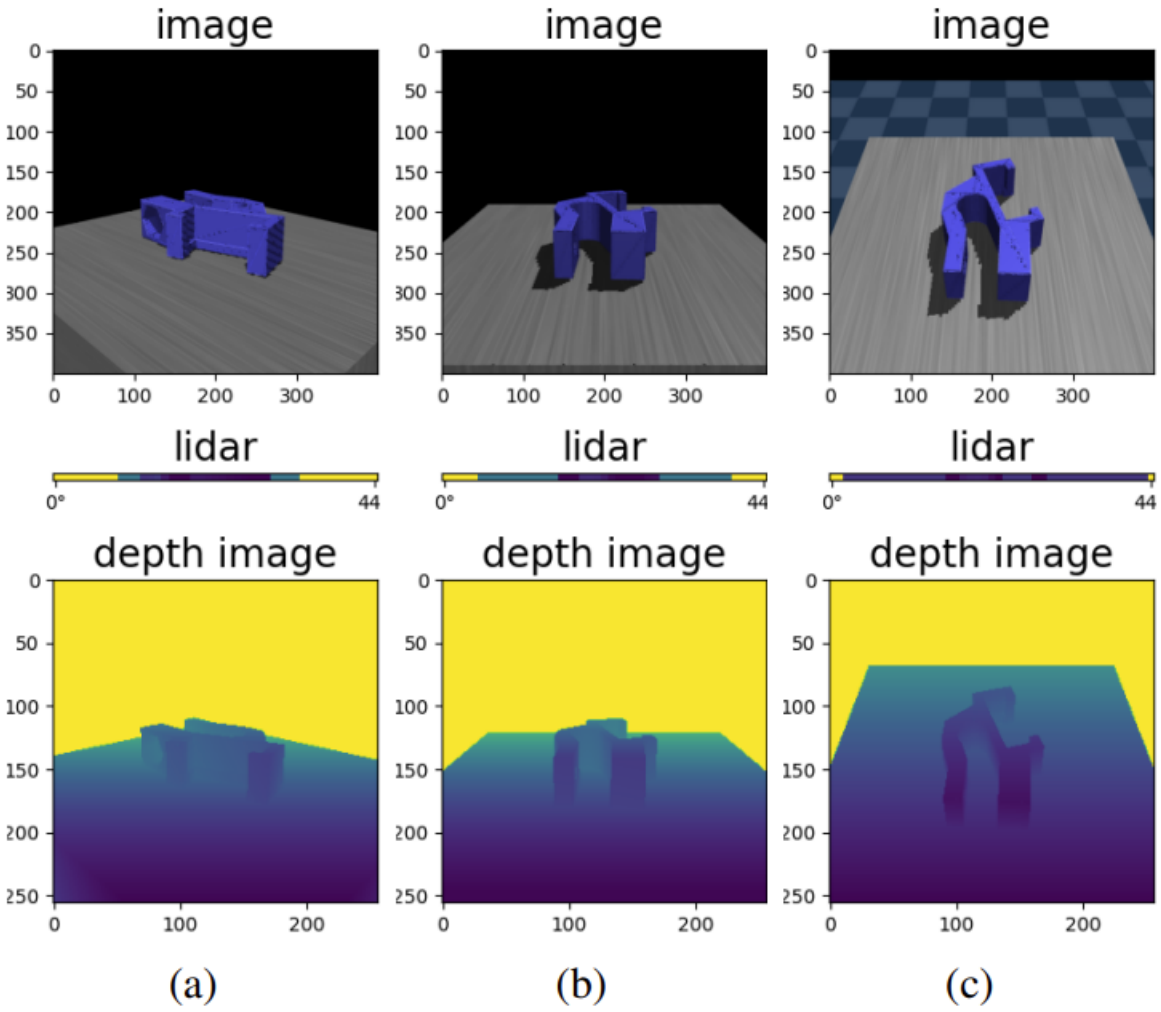


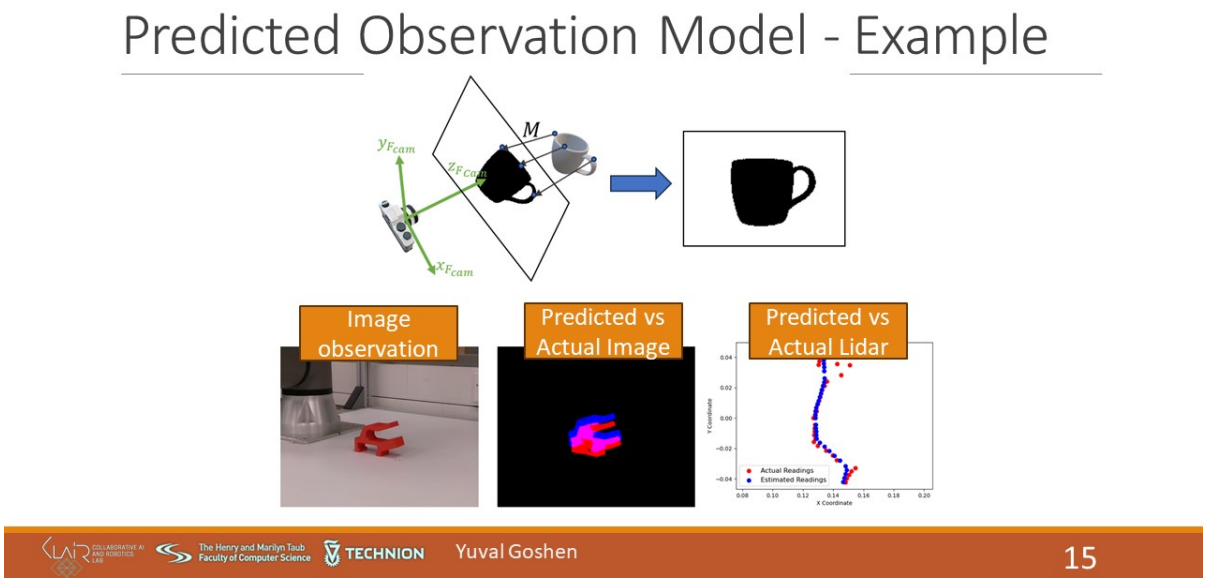
Fig. 4: Example sensor configurations and corresponding observations for a given stable pose. Each column represents the RGB image [top] lidar reading [middle] and depth image

Definition (Sensor Function)

Given object pose $p \in \mathcal{P}_o$ and sensor configuration $q \in \mathcal{Q}$, sensor function $\mathcal{O} : \mathcal{P}_o \times \mathcal{Q} \mapsto \Omega$ is a random function, such that if $o = \mathcal{O}(p, q)$ then $P(o|p, q)$ provides the conditional probability of obtaining the observation o when the sensor configuration is q and the object pose is p.

Typically, the actual distribution is not known, and we use a *predicted sensor function* $\tilde{\alpha}$ and a *predicted observation probability* $\hat{P}$, which may be incorrect or inaccurate.

Challenge: the probability of an observation is near 0.



From Beliefs to Decisions

We use a similarity score, $\omega : \Omega \times \Omega \mapsto [0, 1]$ to compare the predicted and received observations:

$$\hat{P}(o|p, q) = \frac{\omega(\tilde{\alpha}(p, q), o)}{\int_{p' \in \mathcal{P}_o} \omega(\tilde{\alpha}(p', q), o) dp'} \tag{3}$$

The literature is rich of various definitions for ω, which may vary between applications and sensor types.

From Beliefs to Decisions

For **belief update** we use a Bayesian filter such that for any observation $o \in \Omega$ taken from sensor configuration $q \in \mathcal{Q}$, the updated pose belief $\beta^{o,q}(p)$ for pose $p \in \mathcal{P}_o$ is given as

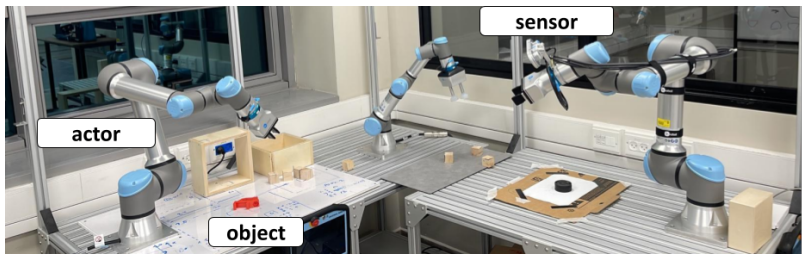
$$\beta^{o,q}(p) = \frac{\hat{P}(o|p,q)\,\beta(p)}{\int_{p' \in \mathcal{P}_o} \hat{P}(o|p',q)\,\beta(p')\,dp'} \tag{4}$$

where $\beta(p)$ is the estimated probability that p is the object pose prior to considering the new observation o.

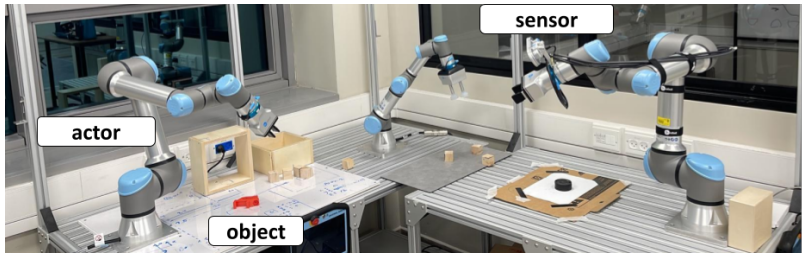
Value of Assistance (VOA) for Grasping

Given the actor's belief $\beta_a \in \mathcal{B}$, the belief of the helping agent $\beta_h \in \mathcal{B}$, the predicated observation probability $\hat{P}$, sensor configuration $q \in \mathcal{Q}$, and the actor's belief update function τ_a,

$$U_\alpha^{VOA}(\beta_h, \beta_{ac}) \stackrel{\text{def}}{=\!=} \tag{5}$$
$$\mathbb{E}_{p\sim\beta_h}\left[\mathbb{E}_{o\sim\hat{P}(o|p,q)}\left[\gamma(q_g^*(\beta_a^{o,q}), p)\right] - \gamma(q_g^*(\beta_{ac}), p))\right].$$



From Beliefs to Decisions





(a) P1 (b) P2 (c) P3 (d) P4 (e) P5 (f) P6

Is this a good observation?